

A3.1_653602

October 23, 2025

1 SVM y multiple testing

Carlos Hernández Márquez

“Doy mi palabra que he realizado esta actividad con integridad académica”

1.1 Revisión del dataset y diferencias iniciales entre clases 2 y 4

Antes de iniciar cualquier análisis, se realizó una revisión general del conjunto de datos `Khan.csv` con el fin de conocer su estructura y asegurarse de que la información estuviera completa.

```
[1]: import pandas as pd

df = pd.read_csv("A3.1 Khan.csv")

df.shape, df.isna().sum().sum(), df.duplicated().sum(), df["y"].value_counts().
    to_dict()
```

```
[1]: ((83, 2309), np.int64(0), np.int64(0), {2: 29, 4: 25, 3: 18, 1: 11})
```

El archivo contiene **83 muestras** y **2309 columnas**, que representan la expresión de distintos genes en cuatro tipos de tejido tumoral.

Al verificar la calidad del conjunto, se observó que no existen valores faltantes ni registros duplicados, lo que permite trabajar con el total de los datos sin necesidad de limpieza adicional.

Con el conjunto validado, el siguiente paso fue explorar las posibles diferencias entre dos de las clases: la **clase 2** y la **clase 4**. Para esto se calculó la diferencia promedio de expresión de cada gen entre ambos grupos.

La intención de este análisis fue obtener un panorama inicial que mostrara qué genes presentan contrastes más marcados en su comportamiento.

El resultado se ordenó de mayor a menor diferencia absoluta, lo que permitió identificar los diez genes con mayor variación entre las dos clases.

```
[2]: X_cols = [c for c in df.columns if c != "y"]
m2 = df.loc[df["y"]==2, X_cols].mean()
m4 = df.loc[df["y"]==4, X_cols].mean()

out = (pd.DataFrame({
```

```

    "delta_mean_2_minus_4": m2 - m4,
    "mean_cls2": m2,
    "mean_cls4": m4
})
.assign(abs_delta=lambda d: d["delta_mean_2_minus_4"].abs())
.sort_values("abs_delta", ascending=False)
.reset_index()
.rename(columns={"index": "gene"}))

top10 = out.head(10)
top10

```

```

[2]:
   gene  delta_mean_2_minus_4  mean_cls2  mean_cls4  abs_delta
0  X187                -3.323151   -1.487488    1.835663    3.323151
1  X509                -2.906537   -0.707556    2.198982    2.906537
2  X2046               -2.424515   -1.711089    0.713426    2.424515
3  X2050               -2.401783   -2.237496    0.164287    2.401783
4   X129               -2.165185   -1.759748    0.405437    2.165185
5  X1645                2.065460    0.932217   -1.133242    2.065460
6  X1319                2.045941    0.464020   -1.581922    2.045941
7  X1955               -2.037340   -0.912812    1.124528    2.037340
8  X1003               -2.011337   -1.807854    0.203483    2.011337
9   X246                1.837830    1.161029   -0.676801    1.837830

```

Al revisar estos valores, se encontró que algunos genes —como *X187*, *X509* y *X2046*— destacan por mostrar diferencias relativamente grandes, del orden de tres unidades.

Esto podría indicar que ciertos genes están más relacionados con los mecanismos que distinguen los tipos de tumor representados por las clases 2 y 4.

Sin embargo, este análisis debe entenderse solo como un primer acercamiento exploratorio.

Las diferencias observadas no significan aún que existan cambios estadísticamente significativos; para confirmarlo, será necesario aplicar pruebas de hipótesis y correcciones por pruebas múltiples, lo cual se abordará en el siguiente punto.

1.2 Pruebas t y corrección por pruebas múltiples

Después de haber identificado los genes con mayores diferencias promedio entre las clases 2 y 4, el siguiente paso fue comprobar si esas diferencias son estadísticamente significativas.

Para hacerlo, se aplicaron **pruebas t de Student** a cada gen, comparando los valores de expresión de las dos clases. En total se evaluaron los 2,308 genes del conjunto, lo que implica que se realizaron el mismo número de pruebas independientes.

Al ejecutar tantas comparaciones al mismo tiempo, existe un riesgo elevado de obtener resultados falsos positivos: es decir, genes que parecen diferentes solo por casualidad.

Para controlar este problema se aplicaron tres métodos de corrección por pruebas múltiples:

- **Bonferroni:** ajusta el nivel de significancia dividiendo entre el número total de pruebas, lo que reduce drásticamente los falsos positivos, aunque a costa de perder sensibilidad.

- **Holm:** ofrece una alternativa menos estricta, manteniendo el control del error familiar pero permitiendo detectar más diferencias reales.
- **Benjamini-Hochberg (FDR):** controla la tasa esperada de falsos descubrimientos, priorizando el equilibrio entre confianza y poder estadístico.

```
[4]: import numpy as np
from scipy.stats import ttest_ind
from statsmodels.stats.multitest import multipletests

X_cols = [c for c in df.columns if c != "y"]
g2 = df.loc[df["y"]==2, X_cols]
g4 = df.loc[df["y"]==4, X_cols]

# p-values por gen (Welch t-test)
tstat, pvals = ttest_ind(g2.values, g4.values, axis=0, equal_var=False,
    ↪nan_policy="omit")

res = pd.DataFrame({
    "gene": X_cols,
    "t_stat": tstat,
    "pval": pvals
})

# Correcciones múltiples

# Bonferroni (FWER)
rej_bonf, p_bonf, _, _ = multipletests(res["pval"].values, alpha=0.05,
    ↪method="bonferroni")

# Holm (FWER)
rej_holm, p_holm, _, _ = multipletests(res["pval"].values, alpha=0.05,
    ↪method="holm")

# Benjamini-Hochberg (FDR)
rej_bh, p_bh, _, _ = multipletests(res["pval"].values, alpha=0.05,
    ↪method="fdr_bh")

res["p_bonf"] = p_bonf
res["p_holm"] = p_holm
res["p_bh"] = p_bh
res["sig_bonf"] = rej_bonf
res["sig_holm"] = rej_holm
res["sig_bh"] = rej_bh

res = res.sort_values("pval", ascending=True).reset_index(drop=True)

summary = pd.Series({
```

```

    "n_genes": len(res),
    "alpha": 0.05,
    "sig_bonf": int(res["sig_bonf"].sum()),
    "sig_holm": int(res["sig_holm"].sum()),
    "sig_bh": int(res["sig_bh"].sum())
})
summary

# Top-10 por método
top10_bonf = res.loc[res["sig_bonf"]].nsmallest(10,
↳ "p_bonf")["gene", "pval", "p_bonf"]
top10_holm = res.loc[res["sig_holm"]].nsmallest(10,
↳ "p_holm")["gene", "pval", "p_holm"]
top10_bh = res.loc[res["sig_bh"]].nsmallest(10,
↳ "p_bh")["gene", "pval", "p_bh"]

```

1.2.1 Bonferroni

```
[6]: top10_bonf.head(10)
```

```
[6]:
```

	gene	pval	p_bonf
0	X1003	4.998692e-17	1.153698e-13
1	X187	3.716887e-16	8.578576e-13
2	X2050	4.084836e-15	9.427801e-12
3	X1955	5.307128e-15	1.224885e-11
4	X1645	8.262889e-15	1.907075e-11
5	X246	1.537507e-14	3.548567e-11
6	X2046	1.769295e-14	4.083533e-11
7	X509	7.555354e-14	1.743776e-10
8	X1954	9.504059e-14	2.193537e-10
9	X1389	6.387904e-13	1.474328e-09

1.2.2 Holm:

```
[7]: top10_holm.head(10)
```

```
[7]:
```

	gene	pval	p_holm
0	X1003	4.998692e-17	1.153698e-13
1	X187	3.716887e-16	8.574859e-13
2	X2050	4.084836e-15	9.419631e-12
3	X1955	5.307128e-15	1.223293e-11
4	X1645	8.262889e-15	1.903770e-11
5	X246	1.537507e-14	3.540879e-11
6	X2046	1.769295e-14	4.072917e-11
7	X509	7.555354e-14	1.738487e-10
8	X1954	9.504059e-14	2.185934e-10
9	X1389	6.387904e-13	1.468579e-09

1.2.3 Benjamini-Hochberg (FDR):

```
[8]: top10_bh.head(10)
```

```
[8]:
```

	gene	pval	p_bh
0	X1003	4.998692e-17	1.153698e-13
1	X187	3.716887e-16	4.289288e-13
2	X2050	4.084836e-15	3.062213e-12
3	X1955	5.307128e-15	3.062213e-12
4	X1645	8.262889e-15	3.814150e-12
5	X246	1.537507e-14	5.833619e-12
6	X2046	1.769295e-14	5.833619e-12
7	X509	7.555354e-14	2.179720e-11
8	X1954	9.504059e-14	2.437263e-11
9	X1389	6.387904e-13	1.474328e-10

Tras realizar las pruebas, los resultados mostraron una notable consistencia entre los métodos.

En los tres casos, los genes **X1003**, **X187**, **X2050**, **X1955**, **X1645**, **X246**, **X2046**, **X509**, **X1954** y **X1389** presentan diferencias de expresión estadísticamente significativas entre las clases 2 y 4.

Estos genes mantuvieron valores p extremadamente bajos, incluso después de aplicar las correcciones más estrictas.

Esto sugiere que las diferencias detectadas son altamente confiables y reflejan variaciones reales en los niveles de expresión entre ambos tipos de tejido.

El método de **Bonferroni** confirmó únicamente estos genes más sobresalientes, lo cual es esperable dado su carácter restrictivo.

El ajuste de **Holm** produjo un conjunto idéntico de genes significativos, mientras que **Benjamini-Hochberg** permitió conservar una sensibilidad similar, manteniendo el mismo grupo como estadísticamente relevante.

1.3 Análisis de varianza (ANOVA) entre las cuatro clases

En esta parte del análisis se buscó ampliar la comparación realizada anteriormente, considerando ahora las **cuatro clases** del conjunto de datos en lugar de solo dos.

El objetivo fue determinar si existen diferencias significativas en la expresión de cada gen entre los distintos tipos de tejido tumoral.

Para lograrlo, se aplicó un **análisis de varianza (ANOVA)** de una vía, el cual permite evaluar si las medias de más de dos grupos difieren de manera estadísticamente significativa.

En este caso, cada gen representa una variable distinta y cada clase (1, 2, 3 y 4) un grupo de comparación.

La prueba se realizó utilizando la función `f_oneway` del módulo `scipy.stats`, que calcula el estadístico F y su correspondiente valor p para cada gen.

Previo a su aplicación, los datos se **estratificaron por clase** para asegurarse de que los valores de cada grupo se analizaran por separado.

Al igual que en el punto anterior, posteriormente se aplicaron correcciones por pruebas múltiples (Bonferroni, Holm y Benjamini-Hochberg) para controlar el error de tipo I y reducir la probabilidad de falsos descubrimientos.

```
[12]: from scipy.stats import f_oneway
from statsmodels.stats.multitest import multipletests

X_cols = [c for c in df.columns if c != "y"]

# Estratificar por clase
groups = [df.loc[df["y"]==k, X_cols].values for k in sorted(df["y"].unique())]

# ANOVA por gen (por columna)
f_stats, pvals = f_oneway(*groups)

anova_res = pd.DataFrame({
    "gene": X_cols,
    "F_stat": f_stats,
    "pval": pvals
})

# Correcciones múltiples
rej_bonf, p_bonf, _, _ = multipletests(anova_res["pval"], alpha=0.05,
    ↪method="bonferroni")
rej_holm, p_holm, _, _ = multipletests(anova_res["pval"], alpha=0.05,
    ↪method="holm")
rej_bh, p_bh, _, _ = multipletests(anova_res["pval"], alpha=0.05,
    ↪method="fdr_bh")

anova_res["p_bonf"] = p_bonf
anova_res["p_holm"] = p_holm
anova_res["p_bh"] = p_bh
anova_res["sig_bonf"] = rej_bonf
anova_res["sig_holm"] = rej_holm
anova_res["sig_bh"] = rej_bh

# Ordenar según menor p-value
anova_res = anova_res.sort_values("pval", ascending=True).reset_index(drop=True)

# Resumen de cuántos genes resultaron significativos por método
summary_anova = pd.Series({
    "n_genes": len(anova_res),
    "sig_bonf": int(anova_res["sig_bonf"].sum()),
    "sig_holm": int(anova_res["sig_holm"].sum()),
    "sig_bh": int(anova_res["sig_bh"].sum())
})
summary_anova
```

```

top10_bonf_anova = anova_res.loc[anova_res["sig_bonf"]].nsmallest(10,
↳ "p_bonf")["gene", "pval", "p_bonf"]
top10_holm_anova = anova_res.loc[anova_res["sig_holm"]].nsmallest(10,
↳ "p_holm")["gene", "pval", "p_holm"]
top10_bh_anova = anova_res.loc[anova_res["sig_bh"]].nsmallest(10,
↳ "p_bh")["gene", "pval", "p_bh"]

```

1.3.1 ANOVA + Bonferroni

```
[13]: top10_bonf_anova.head(10)
```

```
[13]:
```

	gene	pval	p_bonf
0	X1955	1.459035e-24	3.367454e-21
1	X1389	1.772751e-24	4.091510e-21
2	X1003	1.618988e-23	3.736625e-20
3	X2050	4.733702e-22	1.092539e-18
4	X246	6.633722e-22	1.531063e-18
5	X742	2.195548e-21	5.067325e-18
6	X1	3.839240e-20	8.860966e-17
7	X2162	1.035143e-19	2.389109e-16
8	X1954	2.182635e-19	5.037522e-16
9	X1645	2.988392e-19	6.897208e-16

1.3.2 ANOVA + Holm

```
[14]: top10_holm_anova.head(10)
```

```
[14]:
```

	gene	pval	p_holm
0	X1955	1.459035e-24	3.367454e-21
1	X1389	1.772751e-24	4.089737e-21
2	X1003	1.618988e-23	3.733387e-20
3	X2050	4.733702e-22	1.091118e-18
4	X246	6.633722e-22	1.528410e-18
5	X742	2.195548e-21	5.056347e-18
6	X1	3.839240e-20	8.837930e-17
7	X2162	1.035143e-19	2.381863e-16
8	X1954	2.182635e-19	5.020061e-16
9	X1645	2.988392e-19	6.870312e-16

1.3.3 ANOVA + Benjamini-Hochberg (FDR):

```
[15]: top10_bh_anova.head(10)
```

```
[15]:
```

	gene	pval	p_bh
0	X1955	1.459035e-24	2.045755e-21
1	X1389	1.772751e-24	2.045755e-21

2	X1003	1.618988e-23	1.245542e-20
3	X2050	4.733702e-22	2.731346e-19
4	X246	6.633722e-22	3.062126e-19
5	X742	2.195548e-21	8.445542e-19
6	X1	3.839240e-20	1.265852e-17
7	X2162	1.035143e-19	2.986387e-17
8	X1954	2.182635e-19	5.597246e-17
9	X1645	2.988392e-19	6.751740e-17

A diferencia de la prueba *t*, que compara únicamente dos grupos, el ANOVA permite analizar de forma simultánea más de dos categorías, lo que lo convierte en una herramienta más completa para este tipo de estudios.

Entre los genes que mostraron diferencias de expresión más marcadas entre las cuatro clases se encuentran **X1955**, **X1389**, **X1003**, **X2050**, **X246**, **X742**, **X1**, **X2162**, **X1954** y **X1645**.

En todos los casos, los valores ajustados de *p* se mantuvieron extremadamente bajos, del orden de 10^{-1} a 10^{-21} , incluso bajo el método más conservador (Bonferroni).

Esto indica que las variaciones observadas entre las clases no son producto del azar, sino que existen diferencias reales en los niveles de expresión de estos genes a lo largo de las cuatro categorías.

1.4 Modelado con máquinas de soporte vectorial (SVM)

Con el propósito de evaluar la capacidad predictiva de los genes identificados como más relevantes, se construyeron tres modelos de **Máquinas de Soporte Vectorial (SVM)**, cada uno con un tipo de kernel distinto: **lineal**, **polinomial de orden 3** y **radial (RBF)**.

Estos modelos permiten establecer fronteras de decisión que separan las clases de manera óptima en función de las características seleccionadas.

Para reducir el tiempo de cómputo y mantener un enfoque práctico, se empleó únicamente un subconjunto de variables, tomando como base los genes con mayor significancia estadística en los análisis anteriores.

Aunque esta selección introduce un sesgo conocido como *fuga de datos*, se aceptó con fines didácticos, dado que el objetivo del ejercicio es comprender el comportamiento comparativo de los distintos kernels.

Los datos se dividieron en dos conjuntos: uno de **entrenamiento (70%)** y otro de **prueba (30%)**, asegurando que todas las clases estuvieran representadas proporcionalmente.

Antes del entrenamiento, las variables se estandarizaron para que todas tuvieran la misma escala, lo cual es indispensable para el correcto funcionamiento de los SVM.

```
[20]: from sklearn.model_selection import train_test_split
      from sklearn.preprocessing import StandardScaler
      from sklearn.svm import SVC
      from sklearn.metrics import accuracy_score, f1_score, classification_report, \
      ↪confusion_matrix
      import pandas as pd
```



```

# Selección de variables (ejemplo: los 10 genes más significativos del ANOVA)
top_genes = ["X1955", "X1389", "X1003", "X2050", "X246", "X742", "X1", "X2162", "X1954", "X1645"]
X = df[top_genes].copy()
y = df["y"].copy()

# División en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Escalado
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Modelos
svm_linear = SVC(kernel='linear', C=1.0, random_state=42)
svm_poly = SVC(kernel='poly', degree=3, C=1.0, random_state=42)
svm_rbf = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)

# Entrenamiento
svm_linear.fit(X_train_scaled, y_train)
svm_poly.fit(X_train_scaled, y_train)
svm_rbf.fit(X_train_scaled, y_train)

# Predicciones
pred_linear = svm_linear.predict(X_test_scaled)
pred_poly = svm_poly.predict(X_test_scaled)
pred_rbf = svm_rbf.predict(X_test_scaled)

# Evaluación
def eval_model(name, y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred, average='macro')
    print(f"\n{name} → Accuracy: {acc:.3f} | F1-macro: {f1:.3f}")
    print(classification_report(y_true, y_pred))
    print("Matriz de confusión:")
    print(confusion_matrix(y_true, y_pred))

```

1.4.1 SVM Lineal

```
[21]: eval_model("SVM Lineal", y_test, pred_linear)
```

SVM Lineal → Accuracy: 0.960 | F1-macro: 0.971

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

1	1.00	1.00	1.00	3
2	1.00	0.89	0.94	9
3	1.00	1.00	1.00	5
4	0.89	1.00	0.94	8
accuracy			0.96	25
macro avg	0.97	0.97	0.97	25
weighted avg	0.96	0.96	0.96	25

Matriz de confusión:

```
[[3 0 0 0]
 [0 8 0 1]
 [0 0 5 0]
 [0 0 0 8]]
```

1.4.2 SVM Polinomial (grado 3)

```
[18]: eval_model("SVM Polinomial (grado 3)", y_test, pred_poly)
```

SVM Polinomial (grado 3) → Accuracy: 0.920 | F1-macro: 0.927

	precision	recall	f1-score	support
1	1.00	1.00	1.00	3
2	1.00	0.78	0.88	9
3	0.71	1.00	0.83	5
4	1.00	1.00	1.00	8
accuracy			0.92	25
macro avg	0.93	0.94	0.93	25
weighted avg	0.94	0.92	0.92	25

Matriz de confusión:

```
[[3 0 0 0]
 [0 7 2 0]
 [0 0 5 0]
 [0 0 0 8]]
```

1.4.3 SVM RBF

```
[19]: eval_model("SVM RBF", y_test, pred_rbf)
```

SVM RBF → Accuracy: 0.960 | F1-macro: 0.971

	precision	recall	f1-score	support
1	1.00	1.00	1.00	3
2	1.00	0.89	0.94	9
3	1.00	1.00	1.00	5

	4	0.89	1.00	0.94	8
accuracy				0.96	25
macro avg		0.97	0.97	0.97	25
weighted avg		0.96	0.96	0.96	25

Matriz de confusión:

```
[[3 0 0 0]
 [0 8 0 1]
 [0 0 5 0]
 [0 0 0 8]]
```

1.5 Comparación de métricas y elección del mejor kernel

El desempeño de los tres modelos refleja distintos niveles de complejidad en la forma en que separan las clases, y cada uno ofrece una lectura distinta sobre la estructura de los datos.

El **modelo lineal** alcanzó una precisión del 96%, lo que indica que las clases pueden separarse casi perfectamente mediante un hiperplano simple. En este caso, el modelo sugiere que las diferencias entre los tipos de tejido son consistentes y que la información contenida en los genes seleccionados es suficiente para distinguirlos sin necesidad de transformaciones no lineales. Esto tiene sentido en contextos biológicos donde los patrones de expresión entre tipos celulares o tumorales tienden a ser sistemáticos, más que caóticos, y donde las relaciones entre variables suelen tener comportamientos aditivos. En escenarios reales de clasificación genética, un modelo así sería útil por su interpretabilidad, ya que permite observar qué genes contribuyen más a la separación de clases y facilita el estudio de posibles biomarcadores.

El **modelo con kernel polinomial de grado 3** redujo ligeramente su rendimiento, mostrando un 92% de precisión. Este descenso quiere decir que el aumento de complejidad no aportó una ventaja real, e incluso añadió cierta inestabilidad, especialmente al clasificar las muestras de la clase 2. En otras palabras, la introducción de relaciones cúbicas entre las variables parece generar un ajuste excesivo a patrones locales del conjunto de entrenamiento que no generalizan bien en la prueba. En un análisis biológico, un modelo de este tipo podría tener sentido únicamente si se sospecha que existen interacciones entre genes o efectos no lineales relevantes, aunque en este caso, el resultado muestra que no parece necesario complicar el modelo para obtener buenas predicciones.

Por último, el **modelo con kernel radial (RBF)** igualó prácticamente al lineal, con los mismos valores de exactitud y F1. Esto confirma que las clases no requieren fronteras curvas o deformaciones del espacio de predictores para separarse correctamente. La estructura de los datos es lo bastante estable como para que un límite plano capture las diferencias entre los grupos. En contextos más complejos, el kernel RBF suele ser útil cuando los límites entre clases son irregulares o presentan solapamientos; sin embargo, aquí su desempeño equivalente al lineal sugiere que los patrones de expresión son coherentes y bien definidos.